

آرگومان‌های خط فرمان و فایل CSV در ++C

این سند یک رفرنس آموزشی فشرده درباره‌ی آرگومان‌های خط فرمان و کار با فایل‌های CSV در زبان ++C است.

فهرست مطالب

- آرگومان‌های خط فرمان
- تبدیل آرگومان به عدد
- آرگومان‌های نام‌دار (Flags)
- فایل CSV چیست؟
- خواندن فایل CSV

۱. آرگومان‌های خط فرمان

تابع `main` با آرگومان

وقتی یک برنامه را از ترمینال اجرا می‌کنید، می‌توانید اطلاعاتی مستقیماً به آن بدهید. در ++C تابع `main` می‌تواند دو پارامتر بگیرد:

```
int main(int argc, char* argv[])
```

پارامتر	توضیح
<code>argc</code>	تعداد آرگومان‌ها — شامل نام برنامه
<code>argv</code>	آرایه‌ای از رشته‌ها که هر آرگومان را نگه می‌دارد

چاپ همه آرگومان‌ها

```
#include <iostream>
using namespace std;

int main(int argc, char* argv[]) {
    cout << "تعداد آرگومان‌ها: " << argc << endl;

    for (int i = 0; i < argc; i++) {
        cout << "argv[" << i << "] = " << argv[i] << endl;
    }

    return 0;
}
```

اجرا

```
$ ./program hello world 123
```

خروجی

```
argv[0] = ./program argv[1] = hello argv[2] = world argv[3] =
123
```

بررسی تعداد آرگومان

همیشه قبل از استفاده، تعداد آرگومان‌ها را بررسی کنید تا برنامه crash نکند:

```
#include <iostream>
#include <string>
using namespace std;

int main(int argc, char* argv[]) {
    if (argc != 3) {
        cerr << "خطا: باید دقیقاً ۲ آرگومان بدهید" << endl;
        cerr << "<فایل ورودی> <فایل خروجی> " << argv[0] << ": استفاده" << endl;
        return 1;
    }
}
```

```

}

string inputFile = argv[1];
string outputFile = argv[2];

cout << "فایل ورودی: " << inputFile << endl;
cout << "فایل خروجی: " << outputFile << endl;

return 0;
}

```

۲. تبدیل آرگومان به عدد

stoi و **stod** – تبدیل رشته به عدد

آرگومان‌ها همیشه به صورت رشته (`*char`) دریافت می‌شوند. برای تبدیل به عدد از توابع زیر استفاده کنید:

تابع	خروجی
<code>stoi(s)</code>	<code>int</code>
<code>stol(s)</code>	<code>long</code>
<code>stof(s)</code>	<code>float</code>
<code>stod(s)</code>	<code>double</code>

```

#include <iostream>
#include <string>
using namespace std;

int main(int argc, char* argv[]) {
    if (argc < 3) {
        cerr << "<عدد صحیح> <عدد اعشاری> ./program : استفاده" << endl;
        return 1;
    }

    int n = stoi(argv[1]); // string to int
}

```

```
double d = stod(argv[2]); // string to double

cout << "عدد صحیح: " << n << endl;
cout << "عدد اعشاری: " << d << endl;

return 0;
}
```

۳. آرگومان‌های نام‌دار (Flags)

پردازش آرگومان‌های `name value--`

در برنامه‌های واقعی، آرگومان‌ها اغلب به شکل `name value--` یا `n value-` استفاده می‌شوند. با یک حلقه می‌توان همه آرگومان‌ها را بررسی کرد:

```
#include <iostream>
#include <string>
using namespace std;

int main(int argc, char* argv[]) {
    string filename = "";
    bool verbose = false;
    int count = 10;

    for (int i = 1; i < argc; i++) {
        string arg = argv[i];

        if (arg == "--verbose" || arg == "-v") {
            verbose = true;
        } else if ((arg == "--file" || arg == "-f") && i + 1 < argc) {
            filename = argv[++i];
        } else if ((arg == "--count" || arg == "-n") && i + 1 < argc) {
            count = stoi(argv[++i]);
        }
    }
}
```

```
if (verbose) cout << "فعال است verbose حالت" << endl;
cout << "فایل: " << filename << endl;
cout << "تعداد: " << count << endl;

return 0;
}
```

اجرا

```
$ ./program --file data.csv --count 5 --verbose
```

خروجی

```
تعداد: 5 data.csv فعال است verbose حالت
```

۴. فایل CSV چیست؟

Comma-Separated Values

CSV مخفف Comma-Separated Values (مقادیر جداشده با کاما) است. هر خط یک ردیف از جدول است و مقادیر هر ستون با کاما از هم جدا می‌شوند.

نمونه فایل `students.csv`:

```
name,age,grade Ali,20,18.5 Zahra,22,19.0 Reza,21,17.5
```

ویژگی	توضیح
قابل خواندن	هر ویرایشگر ساده‌ای آن را باز می‌کند
سازگاری	Excel، Python، SQL همه پشتیبانی می‌کنند
سبک	فشرده‌تر از XML یا JSON برای داده جدولی
استاندارد	فرمت رسمی تبادل داده (RFC 4180)

۵. خواندن فایل CSV

روش ساده — جداسازی با `getline`

با ترکیب `getline` و `stringstream` می‌توان هر خط را به سلول‌های جداگانه تقسیم کرد:

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
using namespace std;

int main() {
    ifstream file("students.csv");

    if (!file.is_open()) {
        cerr << "خطا: فایل باز نشد" << endl;
        return 1;
    }

    string line;

    while (getline(file, line)) {
        stringstream ss(line);
        string cell;
        vector<string> row;

        while (getline(ss, cell, ',')) // جداسازی با کاما
            row.push_back(cell);

        for (const auto& c : row)
            cout << "[" << c << "]" ";
        cout << endl;
    }

    file.close();
    return 0;
}
```

```
[name] [age] [grade] [Ali] [20] [18.5] [Zahra] [22] [19.0] [Reza] [21] [17.5]
```

خواندن نام فایل از خط فرمان

رایج‌ترین الگو در پروژه‌های واقعی – ترکیب `argc/argv` با خواندن CSV:

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
using namespace std;

void processCSV(const string& filename) {
    ifstream file(filename);

    if (!file.is_open()) {
        cerr << "خطا: نمی‌توان فایل را باز کرد" << filename << endl;
        return;
    }

    string line;
    int rowNum = 0;

    while (getline(file, line)) {
        rowNum++;
        stringstream ss(line);
        string cell;
        vector<string> row;

        while (getline(ss, cell, ','))
            row.push_back(cell);

        cout << "ردیف " << rowNum << ": ";
        for (size_t i = 0; i < row.size(); i++) {
            if (i > 0) cout << " | ";
            cout << row[i];
        }
        cout << endl;
    }
}
```

```

    }

    cout << rowNum << " ردیف خوانده شد." << endl;
    file.close();
}

int main(int argc, char* argv[]) {
    if (argc < 2) {
        cerr << "استفاده: " << argv[0] << " <csv فایل>" << endl;
        return 1;
    }

    processCSV(argv[1]);
    return 0;
}

```

اجرا

```
$ ./program students.csv
```

جدول خلاصه

آرگومان‌های خط فرمان

مفهوم	توضیح
argc	تعداد کل آرگومان‌ها – شامل نام برنامه
argv[0]	نام یا مسیر برنامه
argv[1] به بعد	آرگومان‌های واقعی کاربر
stoi(argv[i])	تبدیل رشته به عدد صحیح
stod(argv[i])	تبدیل رشته به عدد اعشاری
بررسی argc	همیشه قبل از استفاده انجام دهید

